

InvenTrack

Inventory & Order Management System

Project Report

Course: Programming in Java (CSE2006)

Name: Atharav Balaji Khonde

Registration Number: 25BAI10734

Table of Contents

Table of Contents	2
1. Introduction.....	3
2. Problem Statement.....	3
3. Functional Requirements	3
3.1 Module 1 - Product Management (CRUD).....	3
3.2 Module 2 - Stock Transaction Processing	3
3.3 Module 3 - Reporting.....	3
4. Non-Functional Requirements.....	3
5. System Architecture	4
6. Design Diagrams	4
6.1 Use Case Diagram	4
6.2 Workflow Diagram	5
6.3 Class Diagram	6
6.4 Sequence Diagram	7
7. Storage Design	7
7.1 products_db.csv structure.....	7
7.2 orders_db.csv structure	7
8. Design Decisions & Rationale.....	8
9. Implementation Details.....	8
9.1 Product.java.....	8
9.2 ProductService.java	8
9.3 OrderService.java	8
9.4 ReportService.java	8
9.5 Main.java.....	8
10. Multithreading & Concurrency	8
11. Testing Approach	9
12. Screenshots / Results	9
13. Challenges Faced.....	9
14. Learnings & Key Takeaways	10
15. Future Enhancements.....	10
16. References	10

1. Introduction

Many businesses and small shops still mostly manages inventory and stocks manually on notebooks or spreadsheet .This makes tracking quantities, reordering items when running low , maintain stock IN and OUT etc difficult for them. This frequently leads to stockouts, overstocking, and avoidable lost sales.

InvenTrack is a command-line inventory management system built in Java that addresses this problem directly. It allows a user to maintain a product catalog, record stock-in and stock-out transactions, and receive alerts when stock falls below a defined reorder threshold - all without requiring any external database installation. The project was developed as part of the CSE2006 Programming in Java flipped-course evaluation, and applies the core concepts covered across the course syllabus: object-oriented design, custom exception handling, multithreading with synchronization, the Java Collections framework, and file I/O.

2. Problem Statement

Manual or spreadsheet-based inventory tracking is not efficient even for small businesses or inventory: quantities go out of sync with reality, there is no warning when an item needs reordering when it is running low on quantity, and there is no organized history of stock movements. InvenTrack solves this problem by providing a lightweight, offline, terminal-based application that a single user can run to manage products and stock transactions reliably, with built-in validation to prevent inconsistent data (such as negative stock) from ever being recorded.

3. Functional Requirements

The system is organized into three major functional modules, each with a clear input/output structure and a logical workflow for how the user interacts with it:

3.1 Module 1 - Product Management (CRUD)

- Add a new product with name, category, quantity, price, and reorder level
- Show all the products currently stored in inventory
- Update an existing product's details in inventory
- Delete a product in inventory by its id

3.2 Module 2 - Stock Transaction Processing

- Record a stock-in (IN) order, increasing a product's quantity
- Record a stock-out (OUT) order, decreasing a product's quantity
- Reject an OUT order that exceeds available stock, via a custom exception
- Process concurrent orders safely using a lock, so simultaneous transactions on the same product cannot corrupt stock levels

3.3 Module 3 - Reporting

- Generate a low-stock report listing all products at or below their reorder level
- Export the full inventory snapshot to a CSV file for record-keeping

4. Non-Functional Requirements

Requirement	How it is addressed
Performance	File reads/writes are kept minimal per operation; the CSV file is only re-read or rewritten when a CRUD or order operation actually requires it.
Reliability	Every file operation is wrapped in try-catch blocks with meaningful error messages instead of allowing the application to crash.

Usability	A simple numbered CLI menu with clear prompts, and human-readable error messages rather than raw stack traces.
Maintainability	Code is organized into model, service, util, and test packages, each with a single, clear responsibility, making it easy to extend or modify any one module independently.
Security	Basic input validation prevents invalid data (negative quantity/price, empty names) from being persisted.
Scalability	The service layer is decoupled from the storage mechanism, so the underlying storage (currently a CSV file) could be swapped for a database in the future with minimal changes to Main.java or the business logic.

5. System Architecture

InvenTrack follows a simple layered architecture: a CLI entry point (Main.java) that delegates all business logic to three independent service classes, each representing one functional module. The service classes operate on a shared Product model and communicate errors through a single custom exception class. Persistent data is stored in flat CSV files rather than a database, keeping the project runnable with zero external setup.

InvenTrack - System Architecture

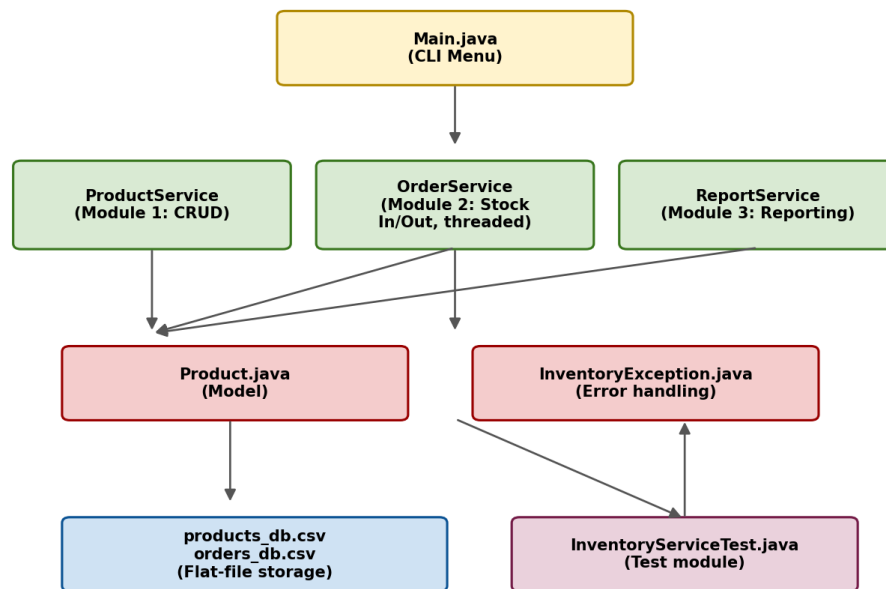


Figure 1: InvenTrack System Architecture

6. Design Diagrams

6.1 Use Case Diagram

The diagram below shows the five primary use cases available to the single actor (User) who interacts with the CLI.

InvenTrack - Use Case Diagram

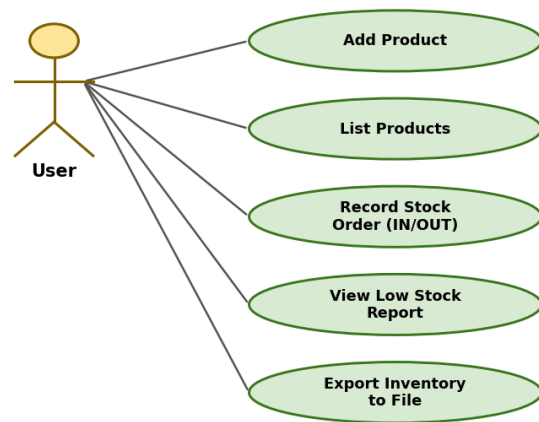


Figure 2: Use Case Diagram

6.2 Workflow Diagram

This diagram illustrates the logical flow of placing a stock order, including validation and the insufficient-stock rejection path.

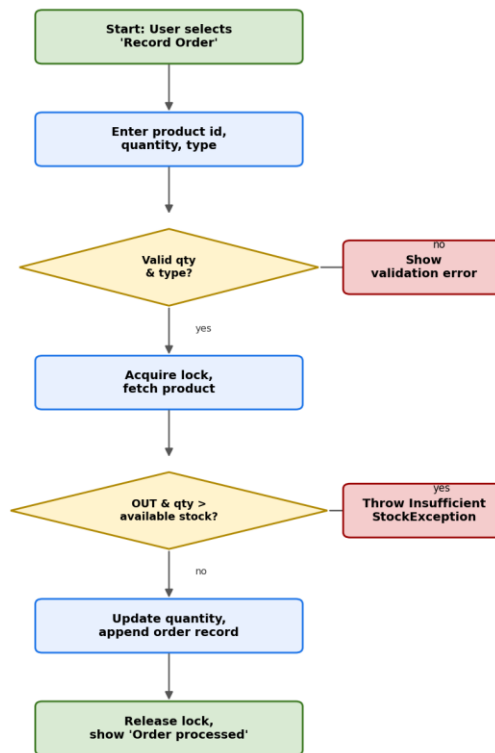
InvenTrack - Stock Order Workflow

Figure 3: Stock Order Workflow Diagram

6.3 Class Diagram

The class diagram shows the main classes, their key fields/methods, and the relationships between them.

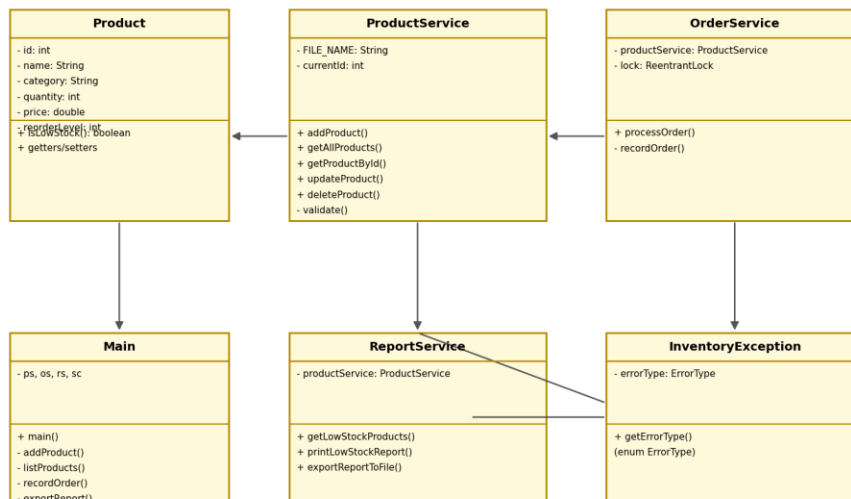
InvenTrack - Class Diagram

Figure 4: Class Diagram

6.4 Sequence Diagram

The sequence diagram traces a single 'place stock-out order' interaction end to end, including the branch where the order is rejected due to insufficient stock.

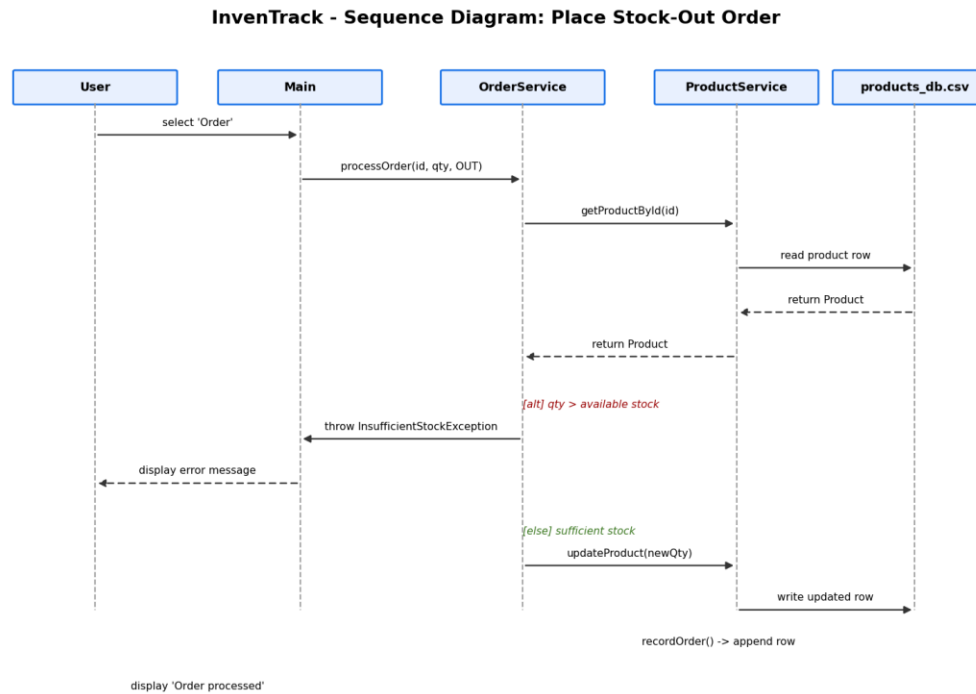


Figure 5: Sequence Diagram - Place Stock-Out Order

7. Storage Design

InvenTrack originally used an SQLite database accessed via JDBC. This was deliberately replaced with flat-file CSV storage to remove the need for evaluators to download and configure a JDBC driver jar before the project can run - the goal being full, zero-setup command-line executability as required by the submission guidelines. The trade-off and rationale are discussed further in Section 8.

7.1 products_db.csv structure

Column	Type	Description
id	int	Unique, auto-incrementing product identifier
name	String	Product name
category	String	Product category
quantity	int	Current stock quantity
price	double	Unit price
reorder_level	int	Threshold below which the product is flagged as low stock

7.2 orders_db.csv structure

Column	Type	Description
product_id	int	Id of the product the order applies to
quantity	int	Quantity moved in this transaction
order_type	String	IN (restock) or OUT (sale/usage)
timestamp	long	Time the order was recorded (epoch milliseconds)

8. Design Decisions & Rationale

- We have used three service classes instead of one they are ProductService, OrderService, and ReportService each maps to exactly one functional module, this keeps responsibilities separate and makes the three-module requirement visually explicit in the codebase.
- We have single custom exception with an ErrorType enum , instead of using one exception class per error case , this will help us in error handling making it consistent and simple to catch in Main.java file, while it still distinguishsh between INSUFFICIENT_STOCK, INVALID_PRODUCT, PRODUCT_NOT_FOUND, and DATABASE_ERROR cases.
- ReentrantLock in OrderService guards the read-check-update sequence in processOrder() so that concurrent stock-out requests on the same product cannot both pass a stock check that only one of them should pass, which would otherwise allow quantity to go negative.
- **We have used csv instead of a database** so the project can be compiled and run with just the standard JDK, without requiring anyone evaluating it to separately download and configure a JDBC driver. This kept setup friction to a minimum for whoever runs it.

9. Implementation Details

9.1 Product.java

A plain model class representing a single product, with an isLowStock() helper method that compares quantity against reorderLevel.

9.2 ProductService.java

Implements Module 1. Reads and writes products_db.csv directly using BufferedReader/BufferedWriter. Validates every product before it is written (rejecting empty names and negative quantity/price), and throws InventoryException with PRODUCT_NOT_FOUND when an operation targets an id that does not exist.

9.3 OrderService.java

Implements Module 2. Uses a ReentrantLock around the get-check-update sequence in processOrder() to make concurrent order processing thread-safe. Successful orders are appended to orders_db.csv for audit history.

9.4 ReportService.java

Implements Module 3. Uses Java Streams (filter/collect) over the product list to compute low-stock items, and writes a full CSV export of the inventory with a LOW/OK status column.

9.5 Main.java

The CLI entry point. Presents a numbered menu, reads user input with Scanner, and delegates every action to the appropriate service. Catches both InventoryException and NumberFormatException so invalid input never crashes the application.

10. Multithreading & Concurrency

To demonstrate the course's multithreading unit, InventoryServiceTest.java includes a dedicated concurrency test that creates a fixed thread pool of 5 threads (via ExecutorService), each attempting to place a simultaneous stock-out order against the same product with limited quantity. Without synchronization, this scenario could allow the total quantity removed to exceed what was actually in stock, since multiple threads could each read the same 'available' quantity before any of them writes an update. The ReentrantLock in OrderService.processOrder() prevents this by ensuring only one thread can execute the read-check-update sequence at a time; threads that arrive while the lock is held simply wait their turn. The test verifies that after all threads complete, the final quantity never goes negative.

11. Testing Approach

InventoryServiceTest.java contains manual test scenarios covering both the happy path and error conditions:

Test	What it verifies
Add valid product	A well-formed product is accepted and persisted
Reject negative quantity	InventoryException is thrown for invalid product data
Reject stock-out exceeding quantity	InventoryException (INSUFFICIENT_STOCK) is thrown rather than allowing quantity to go negative
Concurrent orders	Multiple simultaneous OUT orders on one product never push quantity below zero, verifying the lock in OrderService

Each test prints a PASS or FAIL line to the console, and the suite can be re-run at any time with: `java -cp bin test.InventoryServiceTest`

12. Screenshots / Results

```

C:\Users\athar\OneDrive\Desktop\InvenTrack-Java-Project>cd "c:\Users\athar\OneDrive\Desktop\InvenTrack-Java-Project\src\" && javac Main.java && java Main

1.Add 2.List 3.Order 4.Report 5.Export 6.Exit
Choose: 1
Name: Chips
Category: Estables
Quantity: 2
Price: 20
Reorder level: 1
Product added.

1.Add 2.List 3.Order 4.Report 5.Export 6.Exit
Choose:
  
```

```

1.Add 2.List 3.Order 4.Report 5.Export 6.Exit
Choose: 2
[1] Chips      qty=2      price=20.00

1.Add 2.List 3.Order 4.Report 5.Export 6.Exit
Choose: 3
Product Id: 1
Quantity: 1
Type (IN/OUT): OUT
Order processed.

1.Add 2.List 3.Order 4.Report 5.Export 6.Exit
Choose: 4
=== Low Stock Report ===
Chips      qty=1      reorder=1

1.Add 2.List 3.Order 4.Report 5.Export 6.Exit
Choose:
  
```

```

1.Add 2.List 3.Order 4.Report 5.Export 6.Exit
Choose: 3
Product Id: 1
Quantity: 2
Type (IN/OUT): OUT
Error: Only 1 in stock for Chips

1.Add 2.List 3.Order 4.Report 5.Export 6.Exit
Choose: 5
File path: report.csv
Report exported to report.csv

1.Add 2.List 3.Order 4.Report 5.Export 6.Exit
Choose: 6
Goodbye.

C:\Users\athar\OneDrive\Desktop\InvenTrack-Java-Project>
  
```

```

1.Add 2.List 3.Order 4.Report 5.Export 6.Exit
Choose: 1
Name: COLD DRINK
Category: DRINKABLES
Quantity: 2
Price: 20
Reorder level: 1
Error: please enter a valid number.

1.Add 2.List 3.Order 4.Report 5.Export 6.Exit
Choose:
  
```

13. Challenges Faced

- Balancing course requirements (JDBC, Unit 5) against practical evaluation constraints (zero-setup command-line execution) led to the decision to migrate from SQLite/JDBC to flat-file CSV storage.
- Ensuring thread safety when moving from database-backed storage (which offers some built-in write atomicity) to plain file I/O required careful use of a lock around the entire read-check-update-write sequence in OrderService, rather than assuming file operations alone would be safe under concurrent access.
- Handling malformed user input (e.g., entering text where a number was expected) required explicitly catching NumberFormatException in the CLI loop, in addition to the custom InventoryException, to prevent the application from crashing on bad input.

14. Learnings & Key Takeaways

- I gained Practical experience applying object-oriented design to separate a system into independent, single-responsibility modules.
- I got Hands-on understanding of race conditions in concurrent programs, and how a lock around a critical section prevents them.
- Deeper appreciation for designing a custom exception hierarchy that is expressive without becoming overly fragmented into many small exception classes.
- I Understood the real trade-offs between using a database via JDBC versus simpler file-based persistence, particularly around setup complexity for an end user or evaluator.

15. Future Enhancements

- Reintroduce JDBC-based persistence (e.g., using an embedded database) as an optional storage backend, while keeping the current file-based option for zero-setup use.
- Add a supplier management module to track where stock is reordered from.
- Add unit tests using a proper testing framework such as JUnit, replacing the current manual test harness.
- Add search/filter capabilities (by category, by name) to the Product Management module.

16. References

- Herbert Schildt, "Java: The Complete Reference", 11th Edition, Oracle Press, 2018.
- Oracle Java Documentation - java.util.concurrent package.
- Oracle Java Documentation - java.io package.
- CSE2006 Programming in Java - Course Learning Plan, VIT.